Plugging in the lab's production models, biomass, robust water, and non-linear learners

Salil Gujar
M.Tech CSE Entry No. 2025MCS2106

July 2026

Where we are, and what week 10 tackled

Done so far

- A 4-class base map over India at 10 m with a living hierarchy: split, add, rule-split, merge, and retrain a node on the fly, all rendered as crisp Earth-Engine tiles.
- A git-backed zoo of model and dataset cards, save and resume, a GeoTIFF export, and per-fortnight water.

This week (a chosen subset)

- Plug the lab's own IndiaSAT models straight into the framework as usable, carded classifiers.
- Add non-linear learners: Random Forest on Alpha Earth, XGBoost on Tessera.
- Bring in biomass from GEDI as a regression on the same features.
- Segment the mining class into objects; make water spatially and temporally robust.
- Confirm the STACD implementation against the paper, and measure Tessera against Alpha Earth.

Plugging in the lab's production models (the headline)

The goal here is to use the IndiaSAT models Raman shared: train and store them, and list them in the zoo so a user can pick one to split a class.

- Two models were shared: a pan-India *tree against crop* classifier, and a per-region *farm, plantation, scrubland* classifier.
- The happy discovery: both train a Random Forest *and* classify entirely inside Earth Engine. So, exactly like our base map, they render server-side as crisp tiles, with nothing downloaded and no re-implementation on our side.
- Their training data is readable from our Earth-Engine project, so we reproduce the models faithfully rather than copying weights, which matches how the lab runs them: trained on the fly from a stored ground-truth asset, no saved binary.
- Each is now a runnable overlay on the map and a card in the zoo.

The two IndiaSAT models, and how they fit

- Tree against crop, pan-India. Its features are a Sentinel-1 radar time series, twenty three sixteen-day steps of two polarisations, so it reads how a pixel changes through the year. We build that time series in Earth Engine and classify it with the lab's Random Forest, trained on seventy thousand labelled points.
- Farm, plantation, scrubland, per agro-ecological region. Its features are the same Alpha Earth embedding we already use. For a given area we find its region, train the Random Forest on that region's ground-truth points, and classify. Around one and a half million labelled points back it across nineteen regions.
- Both are cards with a new topology, ee_rf. A card here stores the recipe, the training asset, the feature source, and the class mapping, not a file, because the model is re-trained in Earth Engine on demand.
- The honest caveat: they render as their own overlays and cards now; wiring them in as fully composited split nodes of the hierarchy is a small next step.

Non-linear learners, and where they can run

- The aim was Random Forest on Earth Engine, since it is the model that usually works, and XGBoost on Tessera. XGBoost on Tessera was almost free; Random Forest on Alpha Earth was the real work.
- A Random Forest cannot be replayed as band math, so it cannot ride the crisp tile map. The fix was to make inference *algorithm-aware*: a non-linear split on Alpha Earth is dropped from the tile path and the whole area falls back to the point-grid render, the same path a Tessera split already uses.
- So the user can now choose Random Forest on Alpha Earth; it just renders on the coarser point grid instead of tiles, and the interface says so. Linear models still give the crisp map.
- This also unlocks biomass, which is a Random Forest on the same Alpha Earth features.

Biomass from GEDI, as a regression on our features

- Ratinder's biomass work samples GEDI, a space-borne lidar that measures above-ground biomass, and pairs each shot with the *exact* Alpha Earth embedding we already classify on, plus slope. So biomass is not a new pipeline, it is a regression target on our own feature space.
- We reproduced the data collection over an area and year, with the same quality and error masks, and trained a Random Forest regressor. On his AEZ-8 frame it reproduces his ballpark.
- It rides the non-linear point-grid path from the previous slide, drawn as a green ramp of above-ground biomass in tonnes per hectare, and it is a regression card in the zoo.
- Honest note: biomass from an annual embedding is inherently noisy; a strict region-held-out test scores modestly, which is expected and is the honest number to report.

Segmenting the mining class into objects

- Mining is a per-pixel class today, so it shows as scattered pixels. The goal here is segmentation, the mining area as discrete objects.
- Rather than train a separate segmentation network, which needs imagery and hardware we do not have wired up, we vectorise the existing mining prediction inside Earth Engine: clean the speckle, close pin-holes, trace the connected regions into polygons, and drop anything below a minimum area.
- The result is a handful of clean mining polygons with a per-object area, downloadable as GeoJSON, instead of pixel confetti. On the Asola Bhatti box it returns nine mining segments between half a hectare and one and a half hectares.
- A learned segmentation network remains the future ceiling; this is the pragmatic version that fits the current pipeline.

Water, step one: is it robust across bodies and years

The plan is two steps: first get water against non-water working and see the accuracy, then decide how to fold it into the LULC. This is step one.

- We hold out whole water bodies for a spatial test, and whole years for a temporal test, and the combination for the honest worst case. The seasonal-water polygons carry both a water-body identity and a date, so this needs no new data.
- The within-water-body task is strong and stable: the combined spatial-and-temporal hold-out scores about 0.98, and the year-to-year spread is small, so there is no fluke year.
- This answers the step-one question: the classifier is solid enough within water bodies to build on.

Water, step one: making it work anywhere, and counting fortnights

- The catch: the non-water examples all come from in and around water bodies, so the model over-calls water on ordinary dry land. We augment the non-water class with barren, built-up, and greenery pixels sampled across seasons.
- The effect is exactly as hoped: non-water precision rises from about 0.72 to 0.99, so the model stops painting dry land as water. This augmented, works-anywhere model is now the deployed one.
- We also run the classifier over a whole year of fortnights and count, per pixel, how many fortnights it held water. A perennial body scores high, a monsoon-only pond scores low. This is the layer that would let the LULC tell perennial from seasonal water, the kind of seasonal water the IndiaSAT map is built to catch.
- Folding water into the hierarchy is the step-two decision this first step sets up.

Acacia: spatial and temporal robustness together

Acacia against non-acacia is our hardest split. Last week we showed temporal robustness; this week we add spatial, and combine them.

What is held out	Accuracy
Unseen years only (temporal)	0.716
An unseen region only (spatial)	0.695
An unseen region and unseen years (both)	0.673

The ordering is the honest one: holding out a whole region is harder than random polygons, and holding out region and year together is hardest. The per-year accuracy on the held-out region also flags 2024 as a possible fluke year, the fluke-year check we wanted. The crowns now carry their source region, so this split is possible at all.

How long does Tessera take, against Alpha Earth

Measured live over one small site, so we can advise which pipeline to reach for.

Stage	Tessera	Alpha Earth
Download	about 29 s per tile, 151 MB	none, server-side
Sample	31 s	4 s
Train	under a second	under a second
Classify	40 s, point grid	8 s, tiles
Total	about 72 s	about 12 s

Alpha Earth is server-side round-trips only. Tessera pays a large per-tile download before anything starts, then samples and classifies locally on a grid. So Alpha Earth wins on first touch anywhere; Tessera is worth it only when you need its local features and have already paid the download.

STACD: provenance for every classified output

This week I implemented STACD provenance for our outputs, following the paper's five classes.

- Every classified output emits two things: a stack specification, a STAC Item for the raster with its box, geometry, class legend, and asset links; and a STACD specification, the dependency graph with a dataset and algorithm node per input, one algorithm instance for each live model, rule, and merge, and the output dataset instance.
- The class hierarchy and the ordered operations that built it are embedded as the input set of the producing algorithm, the literal record of what produced this output. This reuses the cards we already keep, so there is no new source of truth and no Earth-Engine work.
- Following the paper closely: each algorithm instance carries a unique identifier, and the output points at a producing *instance* rather than a type. Two naming choices are noted for the paper's authors to confirm.

STACD: the implementation, concretely

What the emitter actually does, all from metadata, no Earth-Engine run.

- The legend is read from the trained models' class lists folded through the active merges, so it mirrors exactly what a classify would paint.
- Each node that resolves its own children becomes an algorithm instance: a trained model, a rule split, or a merge, each with a version and a pointer to where its artifact lives, its code, and its zoo card.
- The input datasets are the Alpha Earth source plus every training dataset any live model consumed, de-duplicated.
- We emit the metadata half of STACD. The runtime half from the paper, an Airflow scheduler with selective recomputation and an instance database, is the natural next layer. This is a shareable record that lines up with the drone and bioacoustics outputs the group already produces.

Where the product stands

- The framework now carries the lab's own production models, not only ones we trained here. That is the piece that turns it from a demo into something the group can actually populate: tree against crop, farm and scrub, biomass, water, all as cards a user can pick.
- Alpha Earth remains the default for browsing anywhere, with linear models on crisp tiles. Random Forest, XGBoost, biomass, and Tessera are there when a task needs them, on the point grid.
- The next step towards a hosted service is packaging, a container and a service-account key. The code is modular, so these sit on top of the existing structure as additions, not rewrites.

Thank you